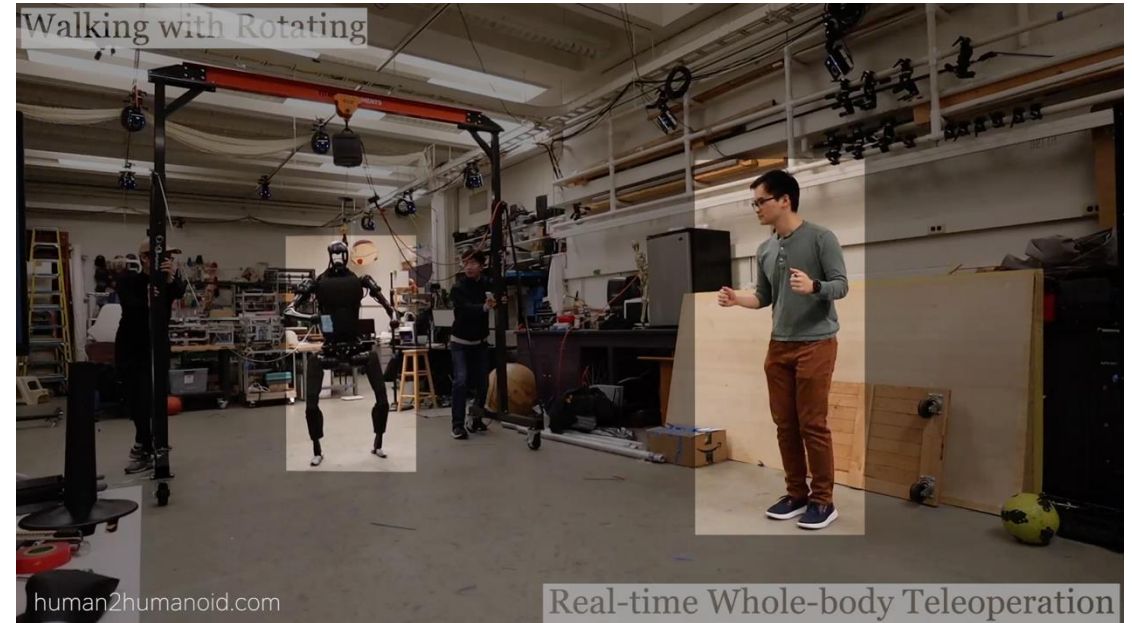
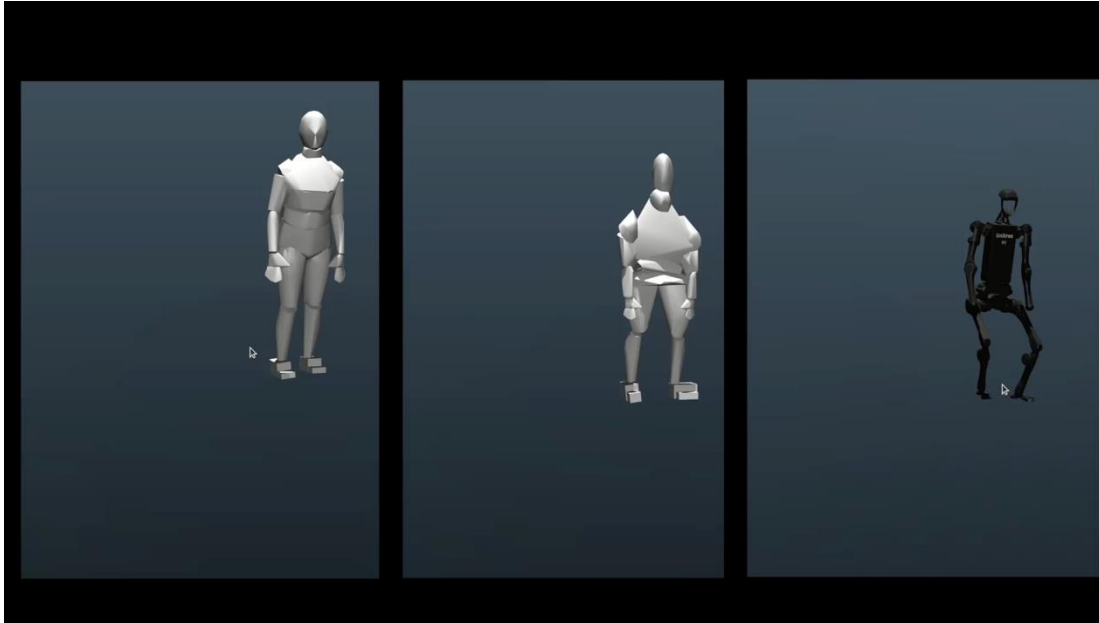


16-831: Introduction to Robot Learning, Fall 2024

Robot Simulation and Sim2Real



<https://human2humanoid.com/>

Instructor: Guanya Shi

Recording Reminder

- ☐ This session will be audio recorded for educational use by other students in this course.

Course Structure

Latest schedule: <https://16-831.github.io/fall24/lectures>

- ☐ Topic group 1: Course introduction: What is robot learning and why?
- ☐ Topic group 2: Machine/Deep learning refresher
- ☐ Topic group 3: Imitation learning
- ☐ Topic group 4: Model-free reinforcement learning
- ☐ Topic group 5: Model-based reinforcement learning
- ☐ Topic group 6: Bandit, preference-based learning, exploration
- ☐ Topic group 7: Reinforcement learning from offline data (one guest lecture)
- ☒ Topic group 8: Specialized topics (one guest lecture)
- ☐ Project presentation

Outline

- ❑ Introduce robot simulators
- ❑ Why is sim2real useful but challenging?

Methods to reduce the sim2real gap:

- ❑ Domain randomization
- ❑ Learning to adapt
- ❑ Learning higher-level policies
- ❑ Real2sim2real

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Outline

- ❑ Introduce robot simulators
- ❑ Why is sim2real useful but challenging?

Methods to reduce the sim2real gap:

- ❑ Domain randomization
- ❑ Learning to adapt
- ❑ Learning higher-level policies
- ❑ Real2sim2real

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

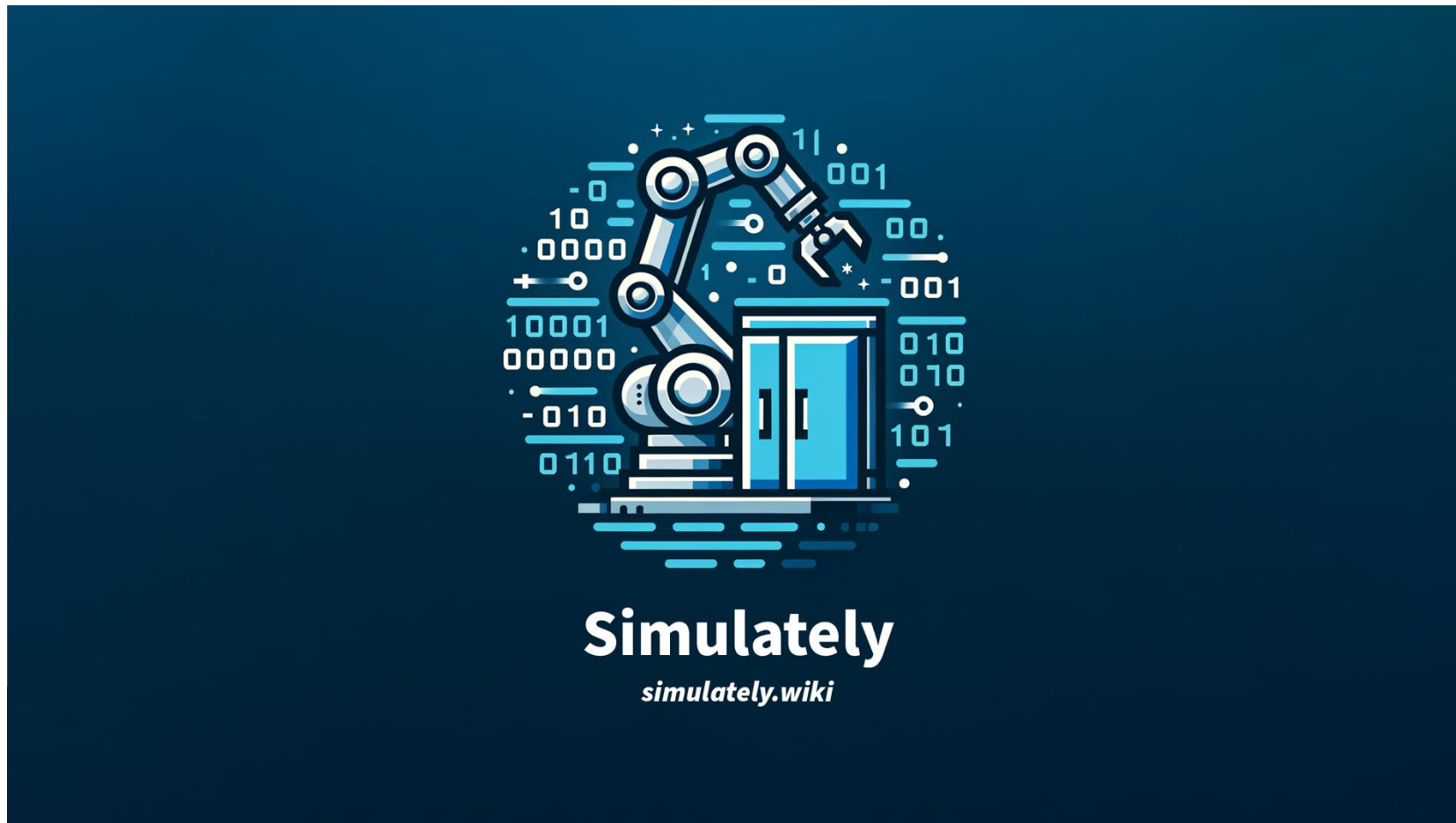
Robot Simulators

- ❑ Thanks to computer graphics and computational physics, there are many simulators!
- ❑ We focus on *physics-based simulators* in this lecture
 - I.e., the simulation process is governed by some *explicit* physical laws
- ❑ Note that there are also non-physics-based simulators (e.g., data-driven like OpenAI Sora)



Robot Simulators

- ❑ Thanks to computer graphics and computational physics, there are many simulators!
- ❑ Simulator taxonomy by simulately.wiki



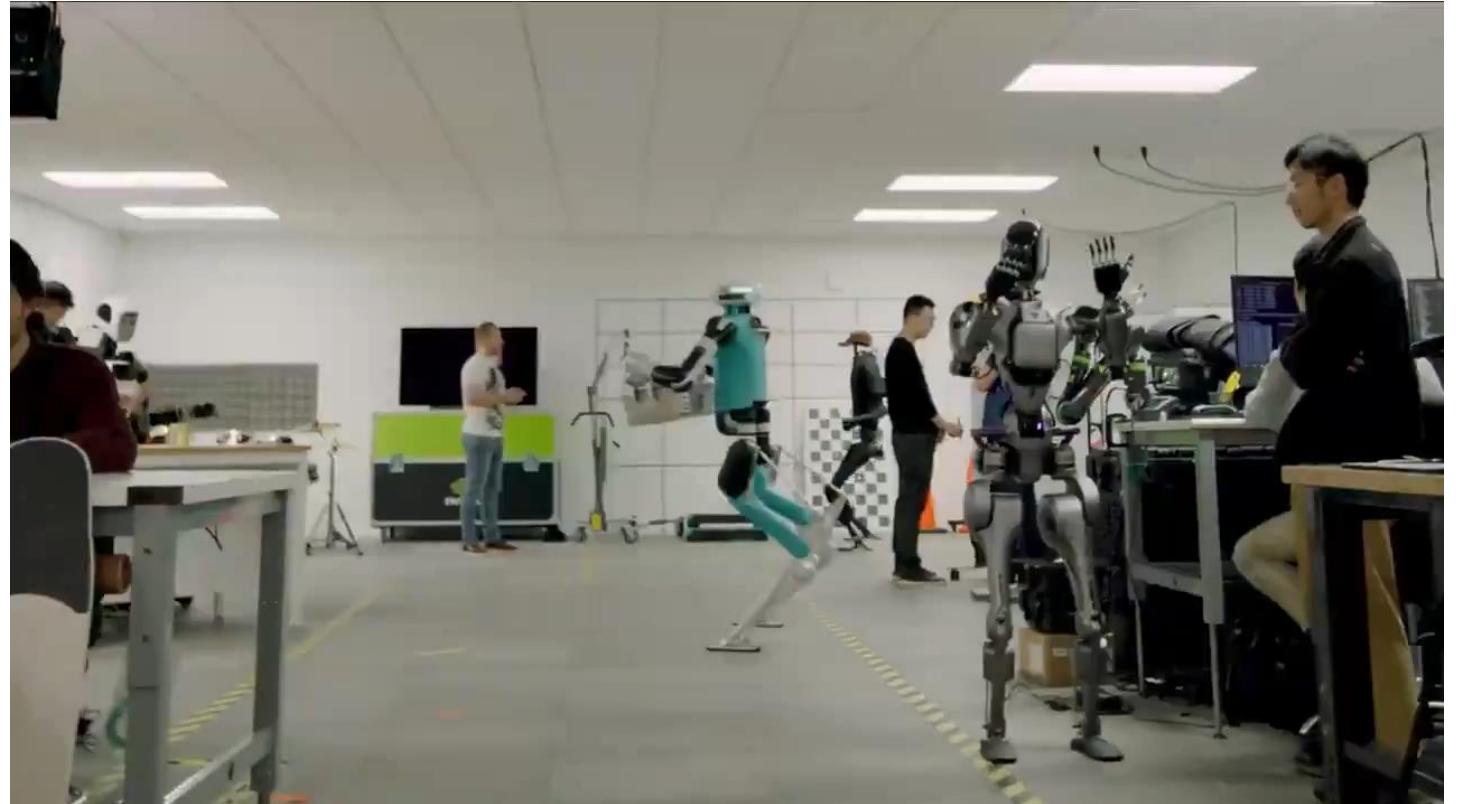
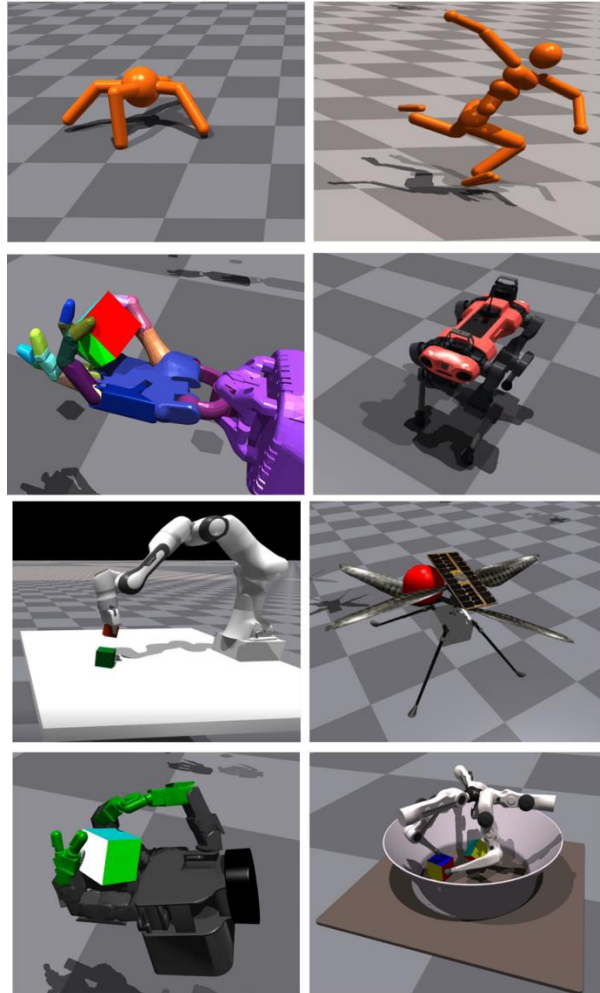
Robot Simulators

❑ Simulator taxonomy by simulately.wiki

Simulator	Physics Engine	Rendering	Sensor 🤖	Dynamics	GPU-accelerated Simulation	Open-Source
IsaacSim	PhysX 5	Rasterization; RayTracing	RGBD; Lidar; Force; Effort; IMU; Contact; Proximity	Rigid;Soft;Cloth;Fluid	✓	✗
IsaacGym	PhysX 5, Flex	Rasterization;	RGBD; Force; Contact;	Rigid;Soft;Cloth	✓	✗
SAPIEN	PhysX 5, Warp	Rasterization; RayTracing ⭐;	RGBD; Force; Contact;	Rigid;Soft;Fluid	✗	✓
Pybullet	Bullet	Rasterization;	RGBD; Force; IMU; Tactile;	Rigid;Soft;Cloth	✗	✓
MuJoCo	MuJoCo	Rasterization;	RGBD; Force; IMU; Tactile;	Rigid;Soft;Cloth	✓💡	✓
CoppeliaSim	MuJoCo; Bullet; ODE; Newton; Vortex	Rasterization; RayTracing 💎;	RGBD; Force; Contact;	Rigid;Soft;Cloth	✗	✓
Gazebo	Bullet; ODE; DART; Simbody	Rasterization;	RGBD; Lidar; Force; IMU;	Rigid;Soft;Cloth	✗	✓

NVIDIA IsaacSim/IsaacGym

- ❑ End-to-end GPU accelerated
- ❑ Massive parallelization of thousands of environments



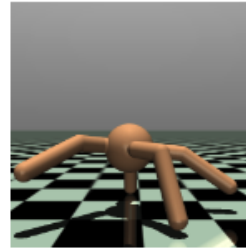
MuJoCo

- ❑ Free and open source!
- ❑ MuJoCo XLA (MJX) allows GPU/TPU accelerations
- ❑ 11 MuJoCo environments/tasks in Gym

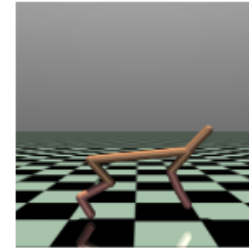
MuJoCo: Multi-Joint dynamics with Contact

[Website](#) [MuJoCo](#) [Doc](#) [MuJoCo](#) [Paper](#) [IEEE](#) [Source](#) [Code](#)

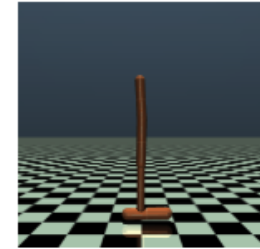
MuJoCo



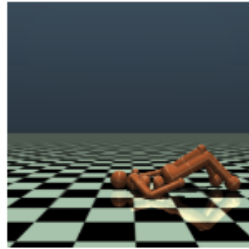
Ant



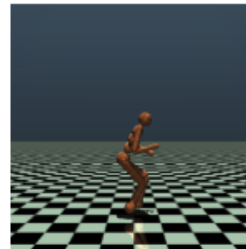
Half Cheetah



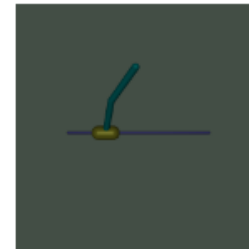
Hopper



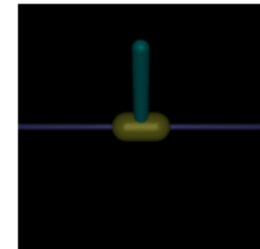
Humanoid Standup



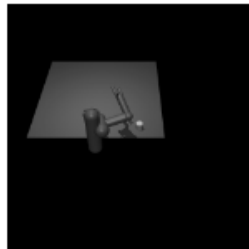
Humanoid



Inverted Double
Pendulum



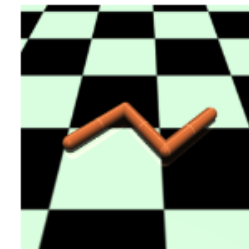
Inverted Pendulum



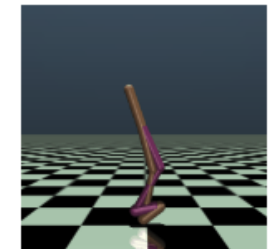
Pusher



Reacher



Swimmer



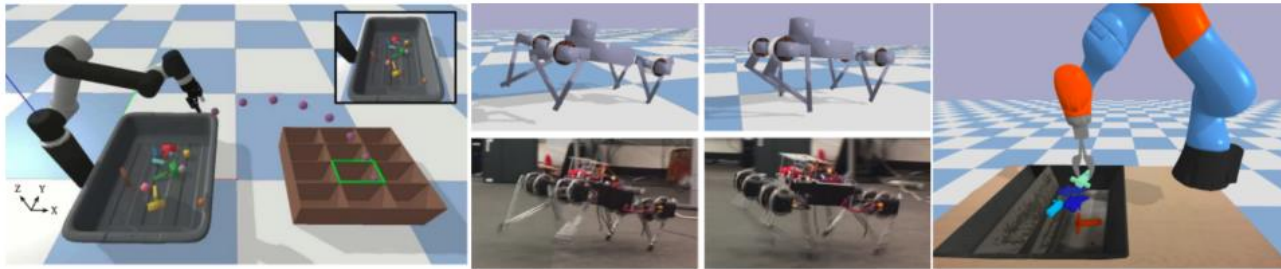
Walker2D

PyBullet

- ❑ Free and open source!
- ❑ Python!

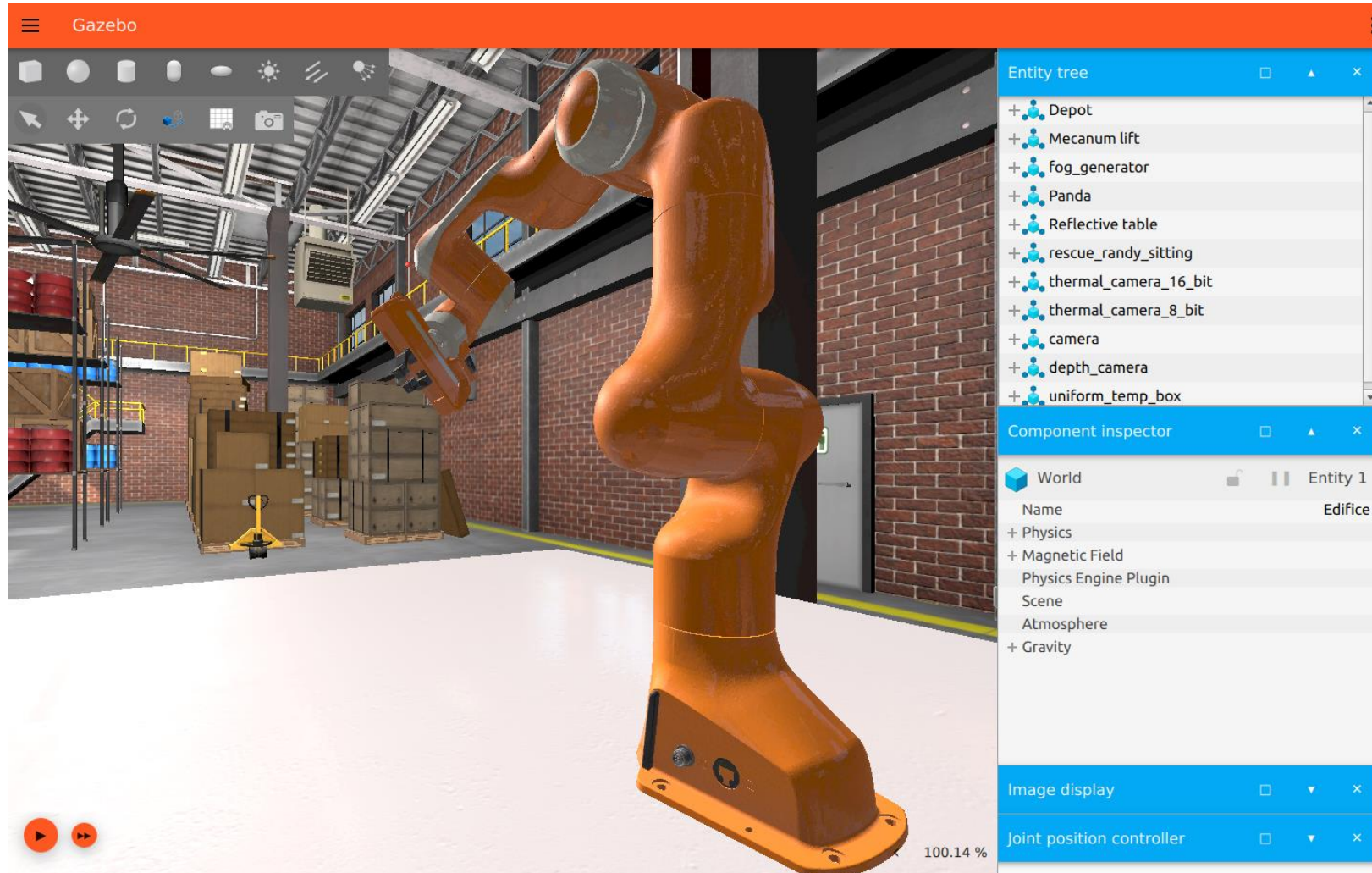
Bullet Real-Time Physics Simulation

[Website](#) [Pybullet](#) [Doc](#) [Pybullet](#) [Source](#) [Code](#)



Gazebo

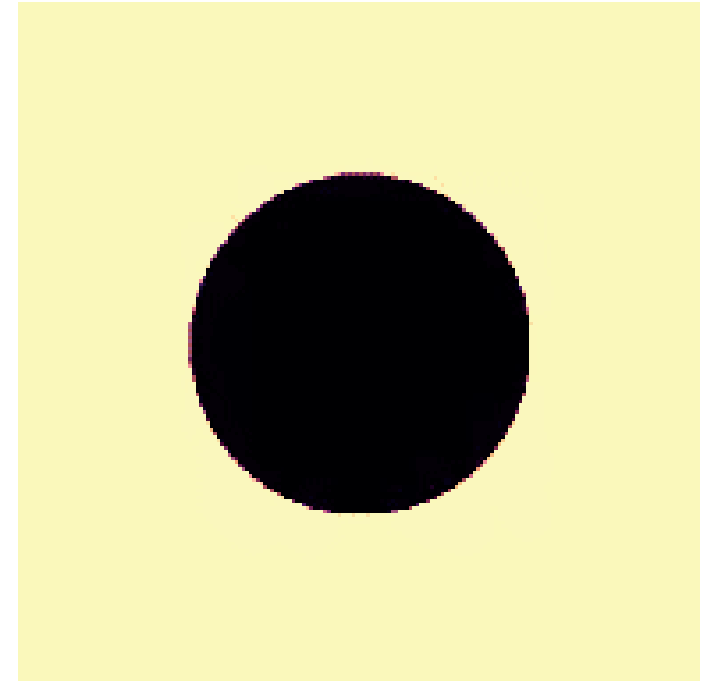
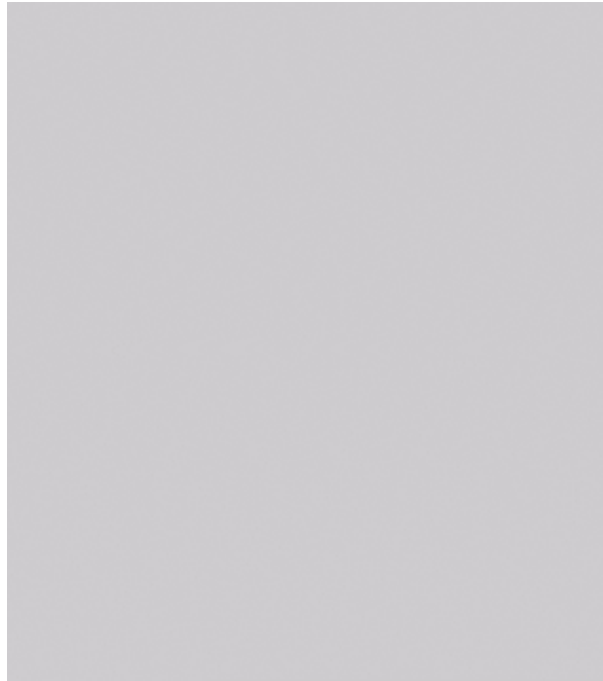
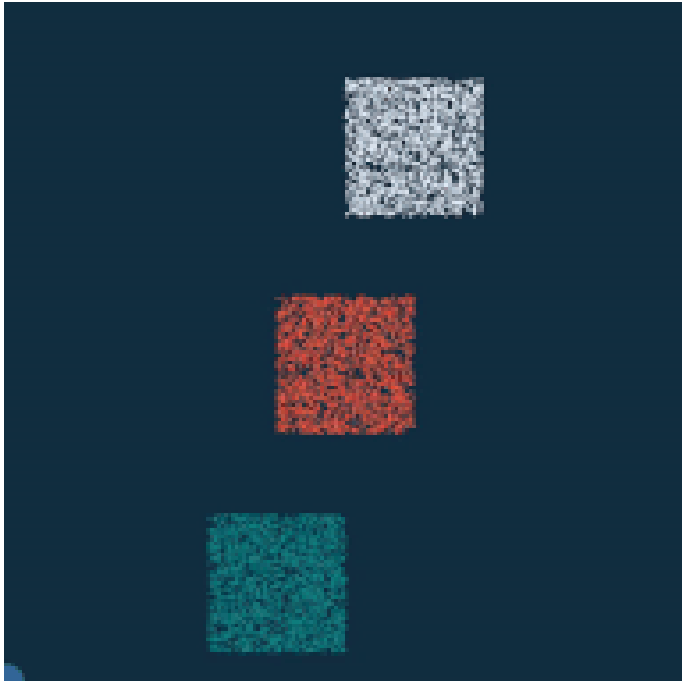
- ❑ Interface nicely with ROS!
- ❑ C++



Beyond Robotics

❑ Simulators are also extremely useful in many other domains. Examples:

- Game, VR/AR, graphics, ...
- Medical
- Manufacture (digital twins)



Outline

- Introduce robot simulators
- Why is sim2real useful but challenging?

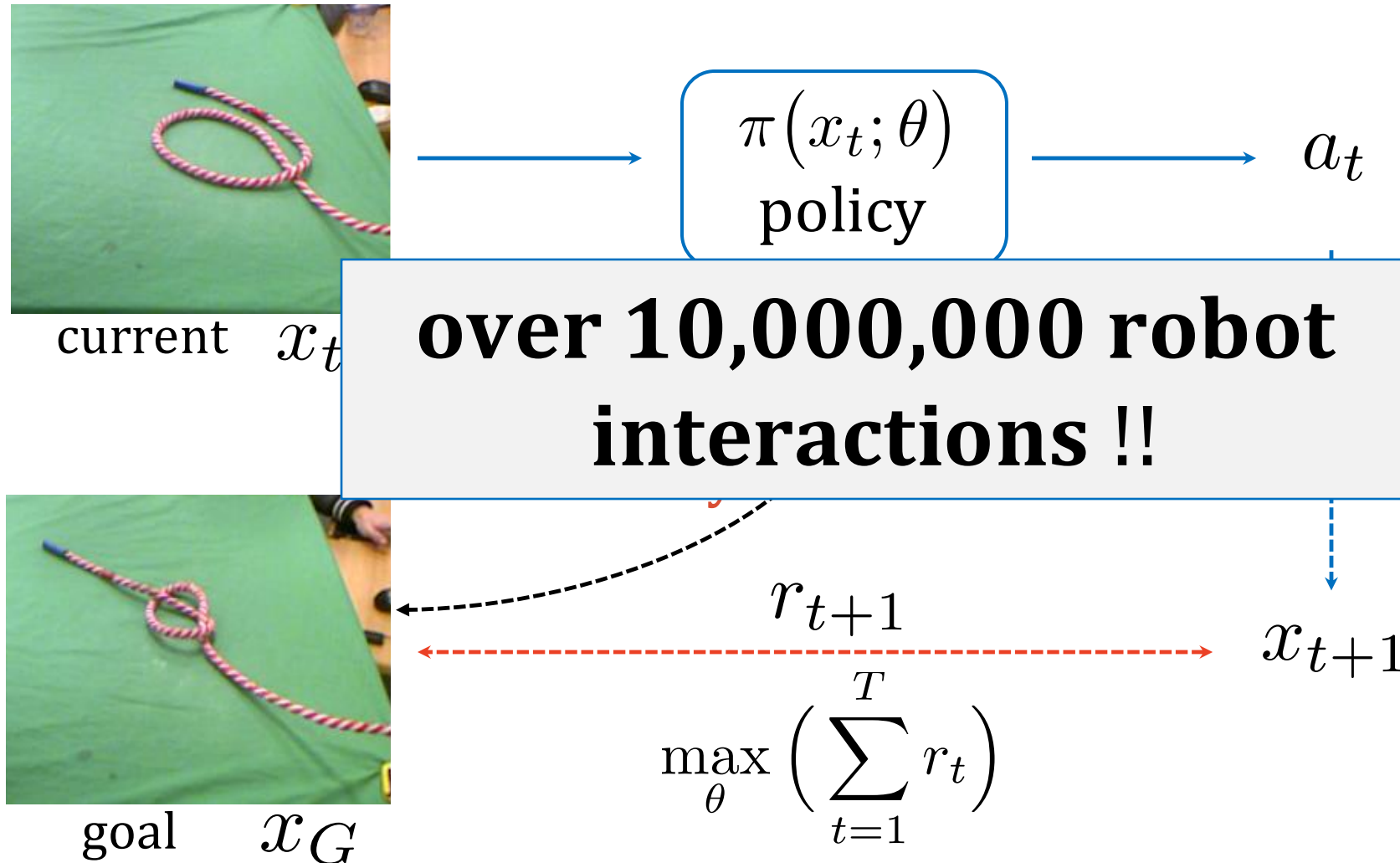
Methods to reduce the sim2real gap:

- Domain randomization
- Learning to adapt
- Learning higher-level policies

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Why Simulators for Robot Learning?

- ❑ Many robot learning algorithms (especially RL-based) are very sample inefficient



Why Simulators for Robot Learning?

Advantages of using simulated data:

- ☐ Cheap, fast, and scalable
- ☐ Safe
- ☐ Labeled (we have access to “oracle” or “ground truth”)
- ☐ Avoid wear and tear of the robot

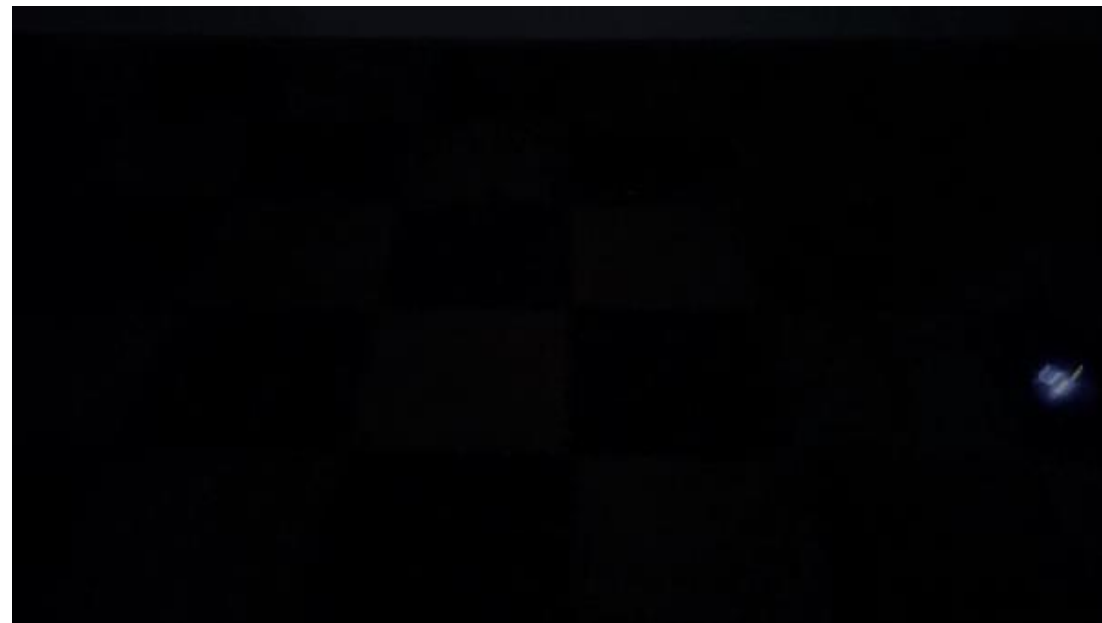
Why Simulators for Robot Learning?

- ❑ Extremely fast and scalable!

Learning to Fly in Seconds

Jonas Eschmann^{1,2}, Dario Albani², and Giuseppe Loianno¹

Simulation. We run our multirotor dynamics simulator on a Nvidia T2000 laptop GPU and attain 1284 million steps/s. To reach this level of performance, 64 blocks of 128 threads are each executing the forward dynamics in parallel. This amounts to 8192 environments in total which are run for 1 000 000 steps each. The required execution time of the GPU kernel is 6380 ms amounting to 1284 million steps/s. At a simulation frequency of 100 Hz this is about 5 months of simulated flight per second. Compared to Flightmare [13] which is the state of the art in terms of dynamics simulation speed with a reported frequency of 200 000 steps/s on a laptop, our simulation is about 6420× faster.

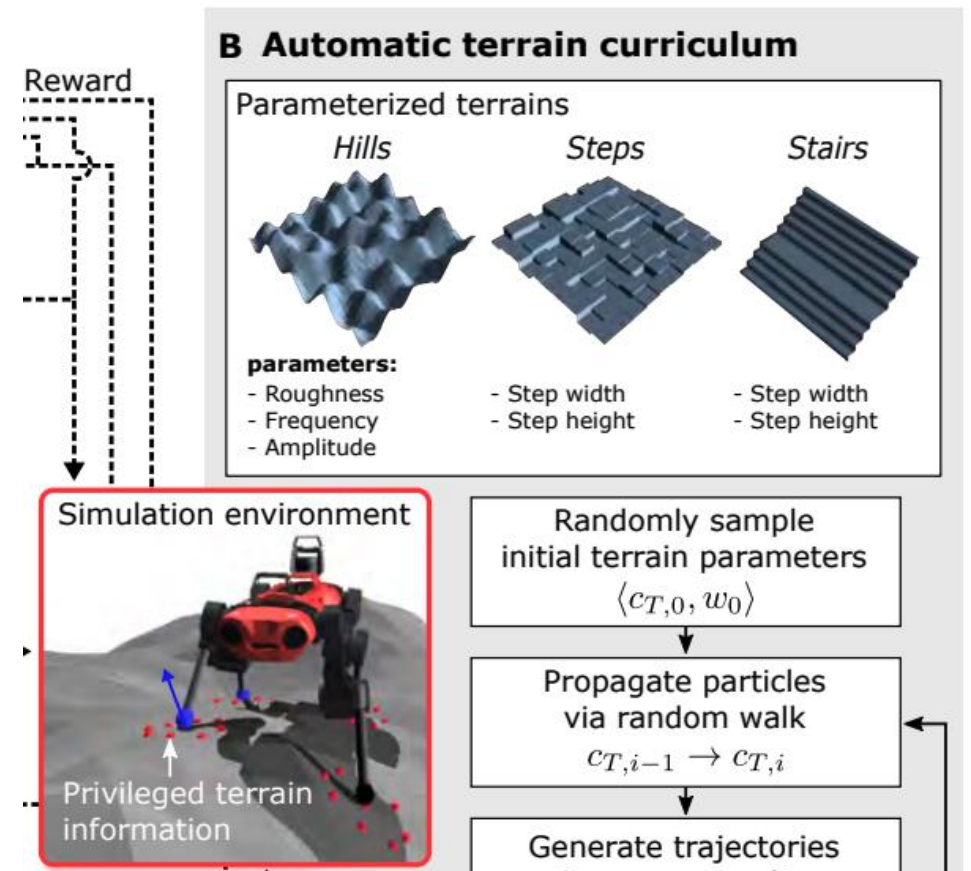
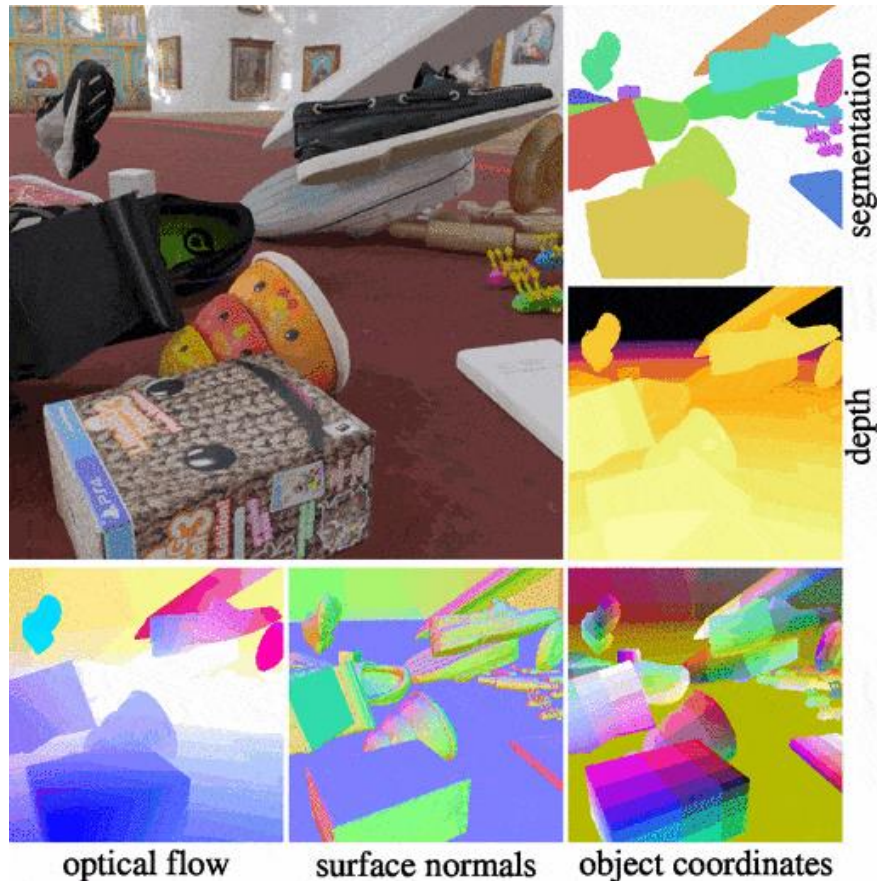


Trained for **18s** on a 2020 MacBook Pro (M1) using RLtools.

Why Simulators for Robot Learning?

- ❑ Get expensive labels for free!

Ground truth in PyBullet



The Problems/Challenges of Sim2Real

Advantages of using simulated data:

- ☐ Cheap, fast, and scalable
- ☐ Safe
- ☐ Labeled (we have access to “oracle” or “ground truth”)
- ☐ Avoid wear and tear of the robot

However, sim2real is never easy!

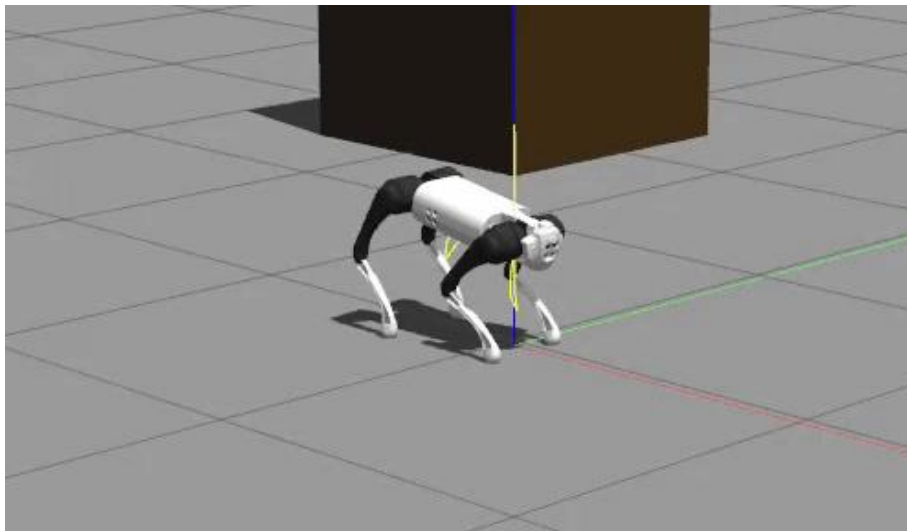
- ☐ It is hard to exactly replicate the real world
- ☐ Non-parametric mismatches (i.e., simulator doesn't consider some effects at all)
- ☐ Parametric mismatches (i.e., simulator uses different parameters than the real)

The Problems/Challenges of Sim2Real

- ❑ Non-parametric mismatches (i.e., simulator doesn't consider some effects at all)
- ❑ Parametric mismatches (i.e., simulator uses different parameters than the real)
- ❑ Recap the MBRL lectures:
 - **Parametric mismatches:** robot mass/inertia, friction, etc
 - Non-parametric mismatches: complex aerodynamics, fluid dynamics, tire dynamics, etc

$$\dot{x} = \frac{dx}{dt} = f(x, u)$$

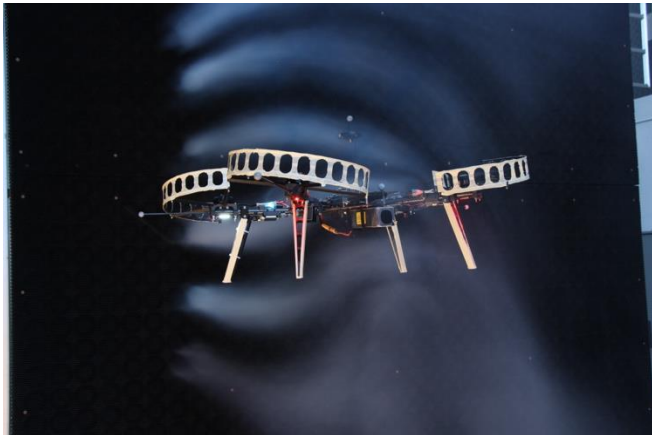
$$x_{t+1} = f(x_t, u_t)$$



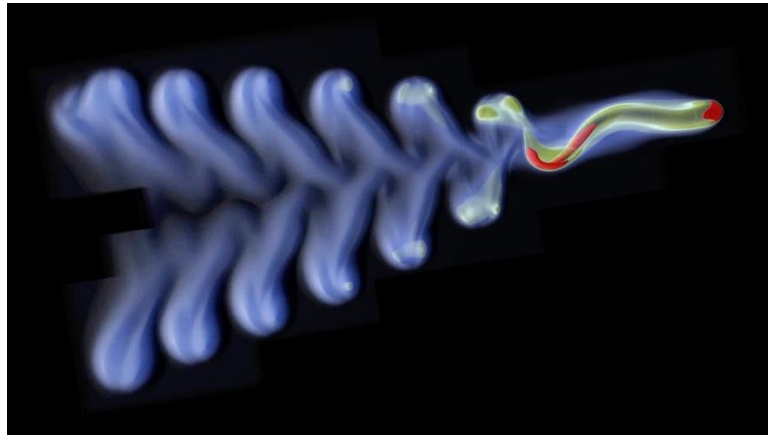
The Problems/Challenges of Sim2Real

- ❑ Non-parametric mismatches (i.e., simulator doesn't consider some effects at all)
- ❑ Parametric mismatches (i.e., simulator uses different parameters than the real)
- ❑ Recap the MBRL lectures:
 - Parametric mismatches: robot mass/inertia, friction, etc
 - **Non-parametric mismatches:** complex aerodynamics, tire dynamics, etc

$$\dot{x} = \frac{dx}{dt} = f(x, u) \quad x_{t+1} = f(x_t, u_t)$$



Aerodynamics in wind, *Neural-Fly*



Fluid dynamics, MIT van Rees Lab



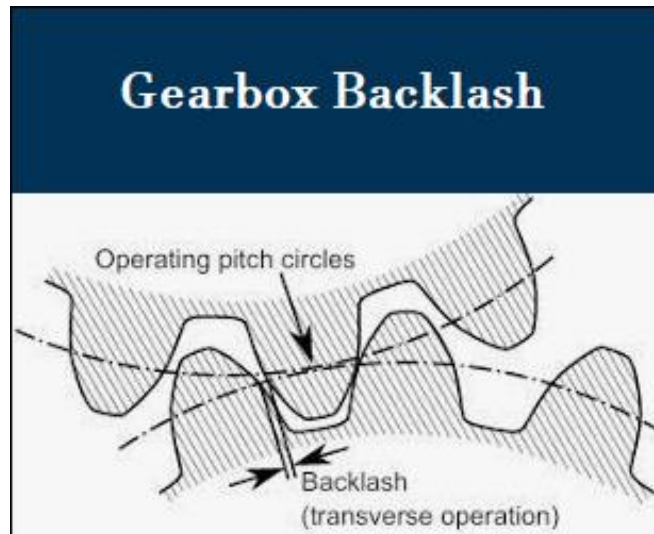
Offroad vehicle dynamics, UW Racer team

The Problems/Challenges of Sim2Real

- ❑ Non-parametric mismatches (i.e., simulator doesn't consider some effects at all)
- ❑ Parametric mismatches (i.e., simulator uses different parameters than the real)
- ❑ Recap the MBRL lectures:
 - Parametric mismatches: robot mass/inertia, friction, etc
 - **Non-parametric mismatches:** complex aerodynamics, tire dynamics, etc



Imperfect modeling of links



Learning quadrupedal locomotion on deformable terrain

Suyoung Choi, Gwanghyeon Ji, Jeongsoo Park, Hyeongjun Kim, Juhyeok Mun, Jeong Hyun Lee, Jemin Hwangbo*



Outline

- ❑ Introduce robot simulators
- ❑ Why is sim2real useful but challenging?

Methods to reduce the sim2real gap:

- ❑ Domain randomization
- ❑ Learning to adapt
- ❑ Learning higher-level policies
- ❑ Real2sim2real

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Domain Randomization

- ❑ Key idea: randomize e in $x_{t+1} = f_{\text{sim}}(x_t, u_t, e)$
 - ❑ Train a *single* RL policy $\pi(x)$ that works for *many* e
 - ❑ An approximation of **robust control**!
-
- ❑ In the original paper, focusing on perception
 - ❑ But this idea has been widely used in all parts in simulation (perception, dynamics, sensor input, delay, etc)

Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World

Josh Tobin¹, Rachel Fong², Alex Ray², Jonas Schneider², Wojciech Zaremba², Pieter Abbeel³

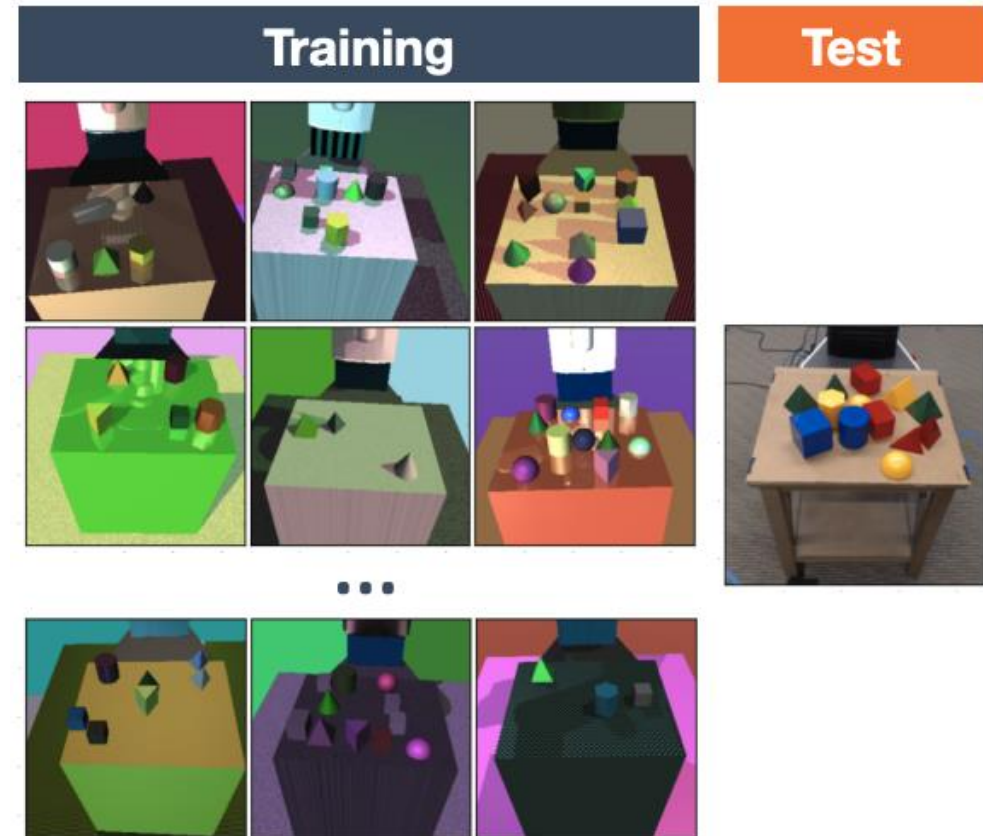


Fig. 1. Illustration of our approach. An object detector is trained on hundreds of thousands of low-fidelity rendered images with random camera positions, lighting conditions, object positions, and non-realistic textures. At test time, the same detector is used in the real world with no additional training.

Domain Randomization Example: Agile But Safe

Tairan He, Chong Zhang, Wenli Xiao, Guanqi He, Changliu Liu, Guanya Shi



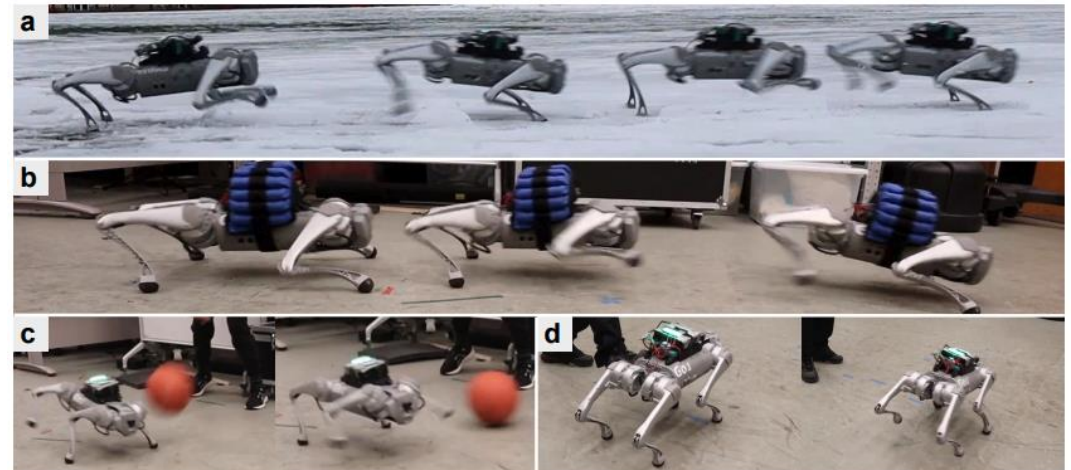
Domain Randomization Example: Agile But Safe

□ Randomize e in $x_{t+1} = f_{\text{sim}}(x_t, u_t, e)$. Train a *single* RL policy $\pi(x)$ that works for *many* e

TABLE II

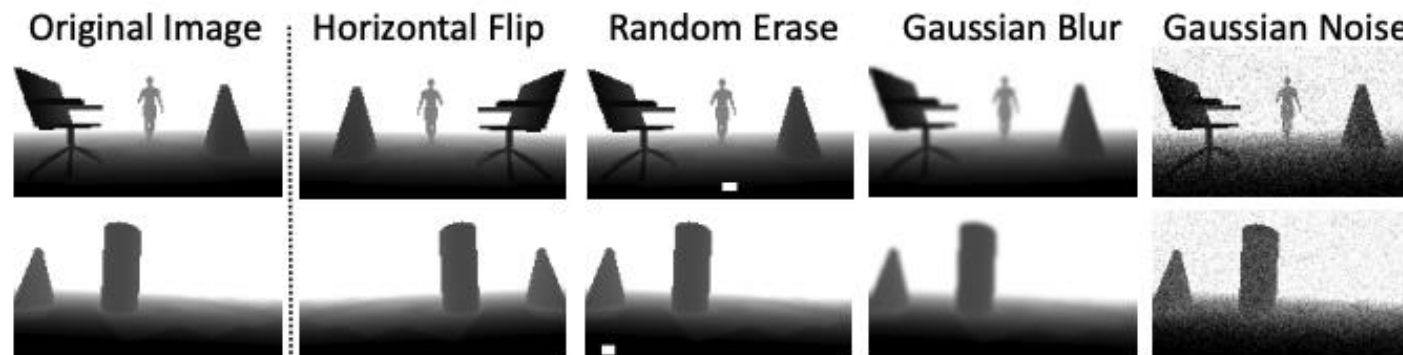
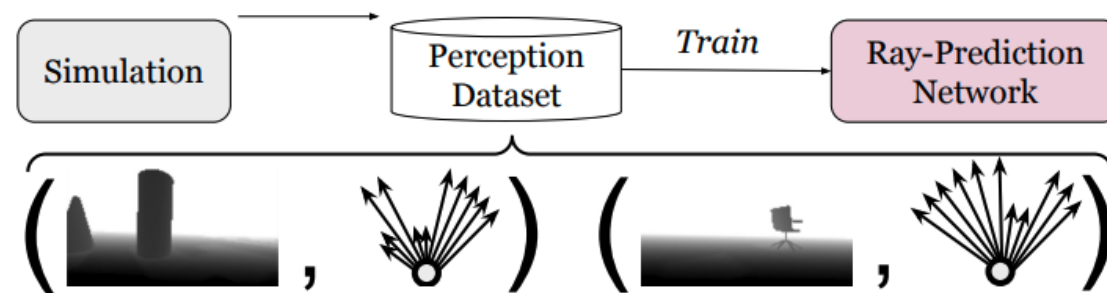
DOMAIN RANDOMIZATION SETTINGS FOR AGILE POLICY TRAINING

Term	Value
Observation	
Illusion	Enabled
Joint position noise	$\mathcal{U}(-0.01, 0.01)$ rad
Joint velocity noise	$\mathcal{U}(-1.5, 1.5)$ rad/s
Angular velocity noise	$\mathcal{U}(-0.2, 0.2)$ rad/s
Projected gravity noise	$\mathcal{U}(-0.05, 0.05)$
log(ray distance) noise	$\mathcal{U}(-0.2, 0.2)$
Dynamics	
ERFI-50 [8]	0.78 N m $\times$ difficulty level
Friction factor	$\mathcal{U}(0.4, 1.1)$
Added base mass	$\mathcal{U}(-1.5, 1.5)$ kg
Joint position biases	$\mathcal{U}(-0.08, 0.08)$ rad
Episode	
Episode length	$\mathcal{U}(7.0, 9.0)$ s
Initial robot position	$x = 0, y = 0$
Initial robot yaw	$\mathcal{U}(-\pi, \pi)$ rad
Initial robot twist	$\mathcal{U}(-0.5, 0.5)$ m/s or rad/s
Goal Position	$x_{\text{goal}} \sim \mathcal{U}(1.5, 7.5)$ m $y_{\text{goal}} \sim \mathcal{U}(-2.0, 2.0)$ m
Goal Heading	$\arctan 2(y_{\text{goal}}, x_{\text{goal}}) + \mathcal{U}(-0.3, 0.3)$ rad



Domain Randomization Example: Agile But Safe

- ❑ Randomize e in $x_{t+1} = f_{\text{sim}}(x_t, u_t, e)$. Train a *single* RL policy $\pi(x)$ that works for *many* e
- ❑ Domain randomization is related to, but different than, *data augmentation* in supervised learning
- ❑ Data augmentation typically refers to augmentation of training data in *supervised learning*
- ❑ Example: augment depth images for ray predictions



Domain Randomization Example: Human2Humanoids



Domain Randomization Example: Human2Humanoids

□ Randomize e in $x_{t+1} = f_{\text{sim}}(x_t, u_t, e)$. Train a *single* RL policy $\pi(x)$ that works for *many* e

Term	Value
Dynamics Randomization	
Friction	$\mathcal{U}(0.2, 1.1)$
Base CoM offset	$\mathcal{U}(-0.1, 0.1)\text{m}$
Link mass	$\mathcal{U}(0.7, 1.3) \times \text{default kg}$
P Gain	$\mathcal{U}(0.75, 1.25) \times \text{default}$
D Gain	$\mathcal{U}(0.75, 1.25) \times \text{default}$
Torque RFI [56]	$0.1 \times \text{torque limit N} \cdot \text{m}$
Control delay	$\mathcal{U}(20, 60)\text{ms}$
External Perturbation	
Push robot	interval = 5s, $v_{xy} = 0.5\text{m/s}$
Randomized Terrain	
Terrain type	flat, rough, low obstacles [57]

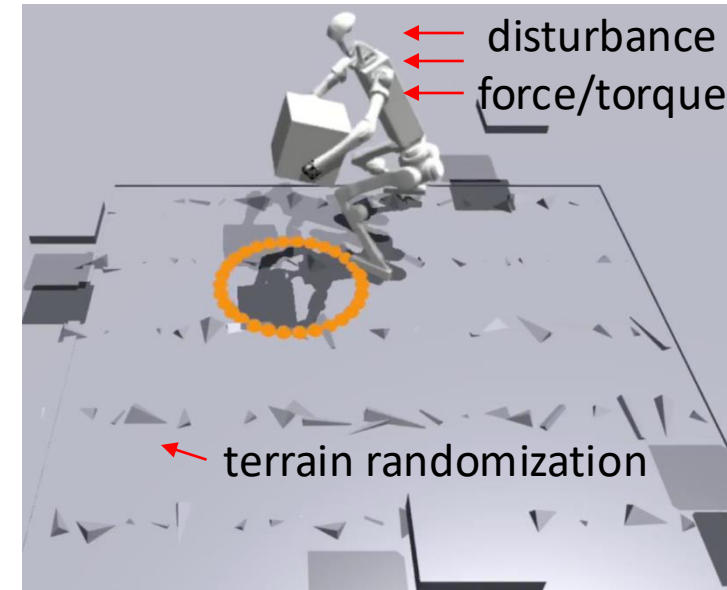


Fig. 7: Robustness Tests of our H2O system under powerful kicking. The policy is able to maintain balance for both stable and dynamic teleoperated motions.

Outline

- ❑ Introduce robot simulators
- ❑ Why is sim2real useful but challenging?

Methods to reduce the sim2real gap:

- ❑ Domain randomization
- ❑ Learning to adapt
- ❑ Learning higher-level policies
- ❑ Real2sim2real

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Learning to Adapt

- ❑ Key idea: randomize e in $x_{t+1} = f_{\text{sim}}(x_t, u_t, e)$
- ❑ Train an *adaptive* RL policy $\pi(x, e)$ that works for *many* e
- ❑ An approximation of **adaptive control**!
- ❑ It doesn't conflict with domain randomization. You can do both (robust adaptive control)!

- ❑ The problem of $\pi(x, e)$: e is often *unknown* in the real!
- ❑ Therefore, a common pipeline for learning to adapt: *learning from a privileged teacher*:
 - Sim: First train a teacher policy with privilege information $\pi(x, e)$
 - Sim: Then a student policy $\pi_s(x, \text{available info in the real})$ learns from $\pi(x, e)$
 - Real: Then deploy $\pi_s(x, \text{available info in the real})$ in the real
- ❑ The second phase is an imitation learning problem (covered in Lecture 6)

Learning to Adapt Example: Learning by Cheating

- ❑ Stage 1: learn a “privileged agent” from expert
 - It knows ground truth state (traffic light, other vehicles’ pos/vel, etc)
- ❑ Stage 2: a sensorimotor student learns from this train privileged agent

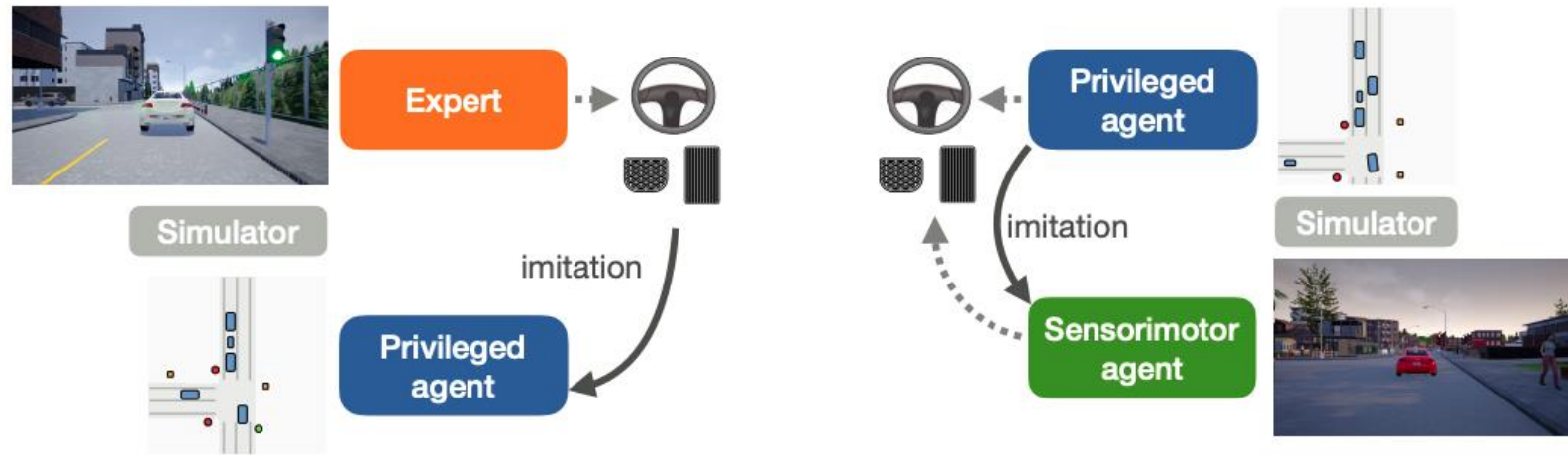
Learning by Cheating

Dian Chen
UT Austin

Brady Zhou
Intel Labs, UT Austin

Vladlen Koltun
Intel Labs

Philipp Krähenbühl
UT Austin



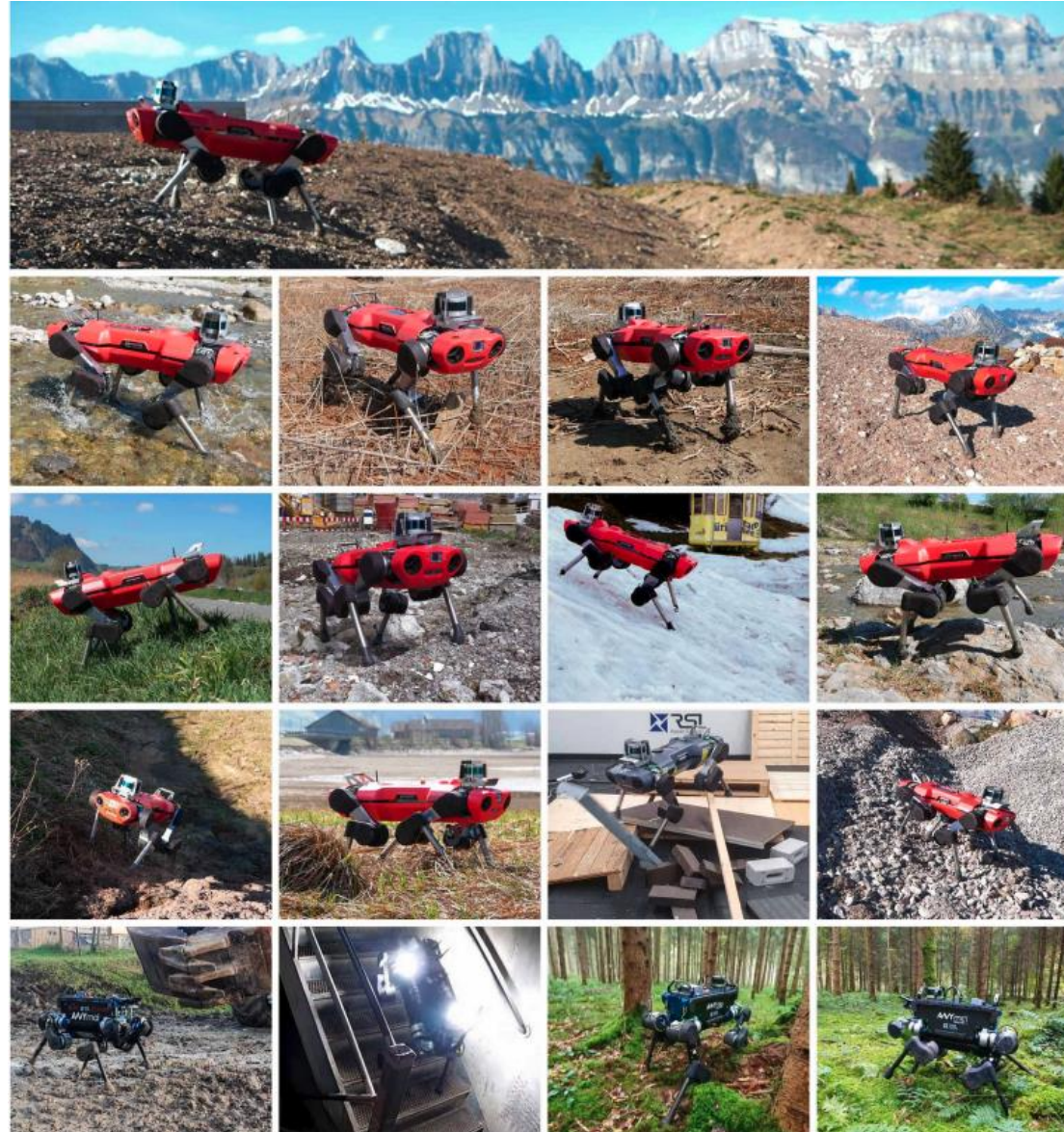
(a) Privileged agent imitates the expert

(b) Sensorimotor agent imitates the privileged agent

Learning to Adapt Example: Locomotion

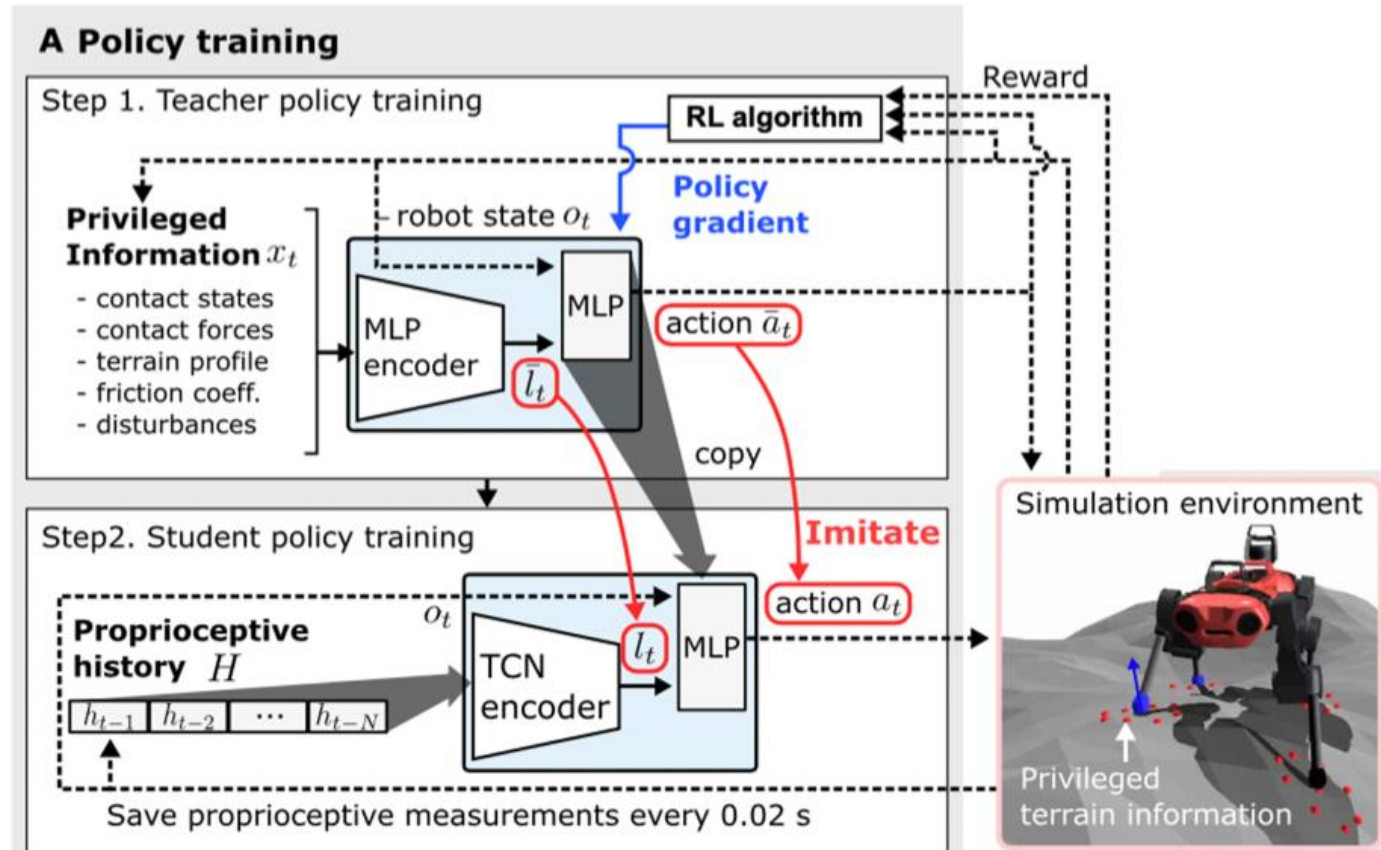


Learning to Adapt Example: Locomotion



Learning to Adapt Example: Locomotion

- ❑ Privileged information contains almost everything available in sim (contact, terrain, friction, disturbance, etc)
- ❑ Students only learn from proprioceptive history (IMU, joint angle, etc)
- ❑ Privileged teacher is trained using RL (PPO, different from the driving example)



Some Variants: Student Learning Not in Action Space

- ❑ Student learning doesn't have to happen in the action space
 - RMA: in latent space

RMA: Rapid Motor Adaptation for Legged Robots

Ashish Kumar
UC Berkeley

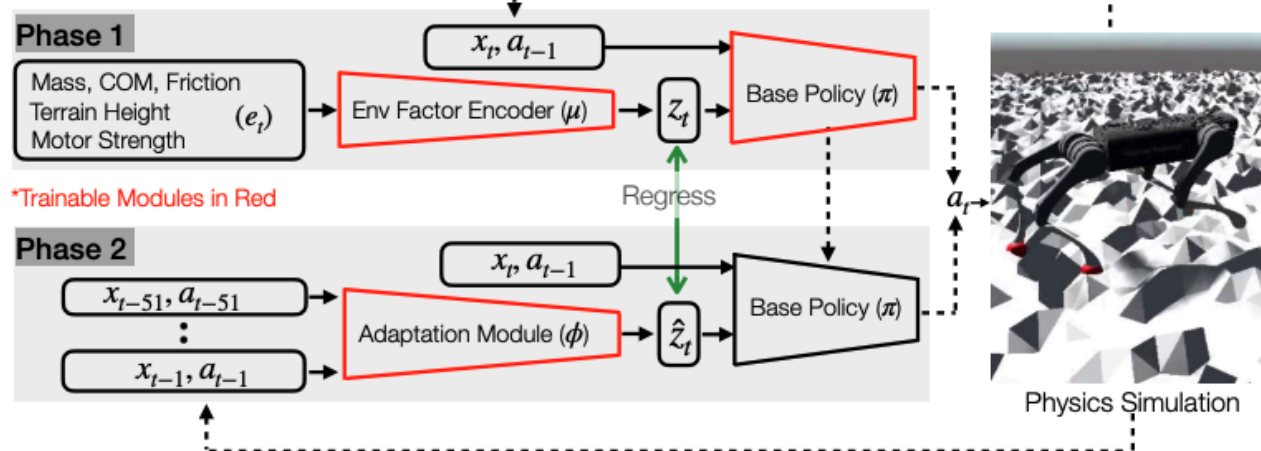
Zipeng Fu
Carnegie Mellon University

Deepak Pathak
Carnegie Mellon University

Jitendra Malik
UC Berkeley, Facebook

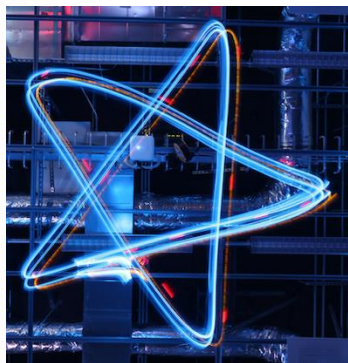
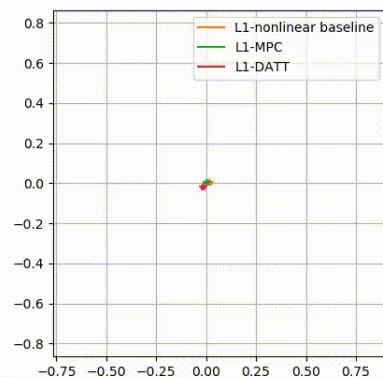


A) Training in Simulation

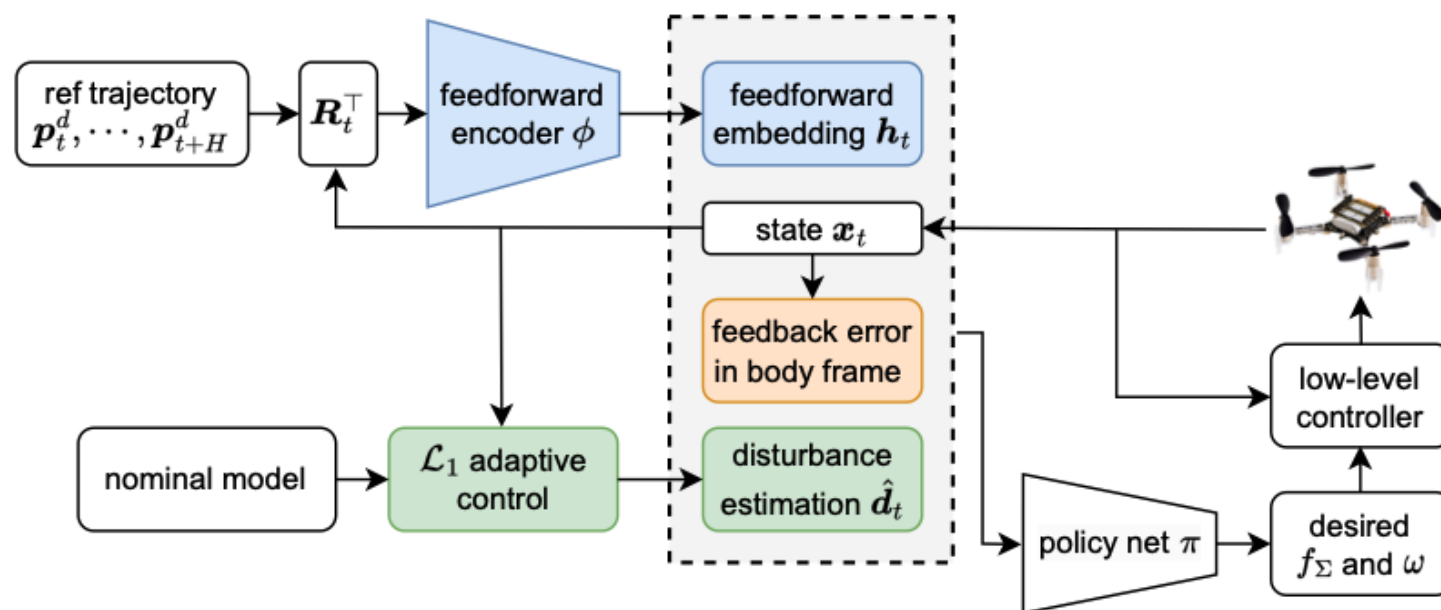


Some Variants: Student Learning Not in Action Space

- ❑ Student learning doesn't have to happen in the action space
 - DATT: student predicts unknown dynamics



$$\begin{aligned}\dot{\mathbf{p}} &= \mathbf{v} \\ m\dot{\mathbf{v}} &= m\mathbf{g} + \mathbf{R}\mathbf{e}_3 f_{\Sigma} + \mathbf{d} \\ \dot{\mathbf{R}} &= \mathbf{R}S(\boldsymbol{\omega}) \\ \mathbf{J}\dot{\boldsymbol{\omega}} &= \mathbf{J}\boldsymbol{\omega} \times \boldsymbol{\omega} + \boldsymbol{\tau}\end{aligned}$$



Outline

- ☐ Introduce robot simulators
- ☐ Why is sim2real useful but challenging?

Methods to reduce the sim2real gap:

- ☐ Domain randomization
- ☐ Learning to adapt
- ☒ Learning higher-level policies
- ☐ Real2sim2real

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

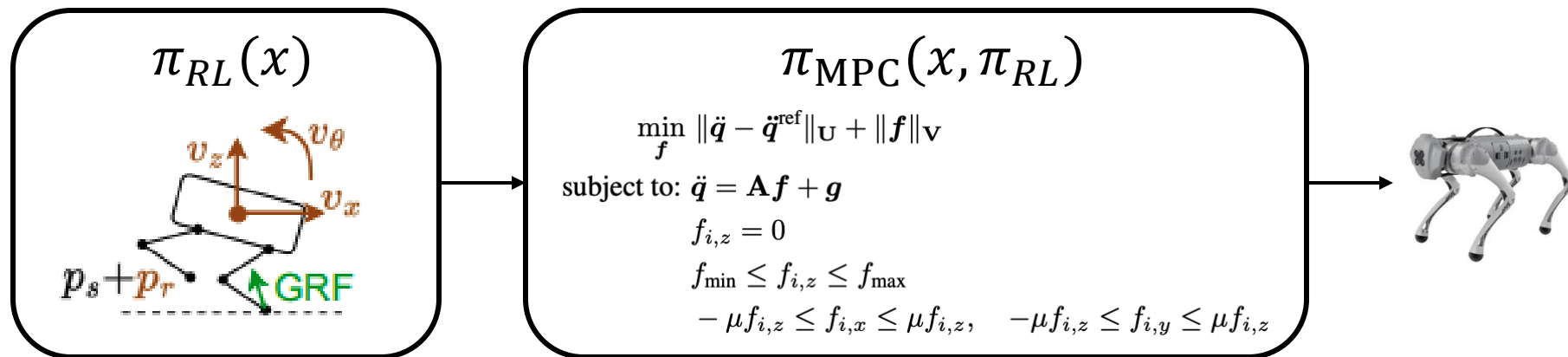
Learning Higher Level Policies

- ❑ Abstraction: Learning higher level policies, not low-level controllers
- ❑ Because the low-level dynamics are very different between simulation and reality



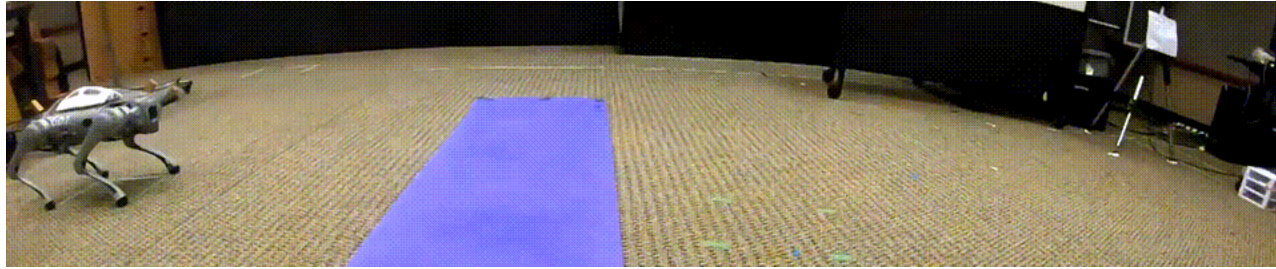
Learning Higher Level Policies

- ❑ High-level centroidal RL outputs:
 - Desired base velocity
 - Gait frequency
 - Swing leg residual trajectory
- ❑ Low-level MPC:
 - Optimize ground reaction force (GRF) using QP



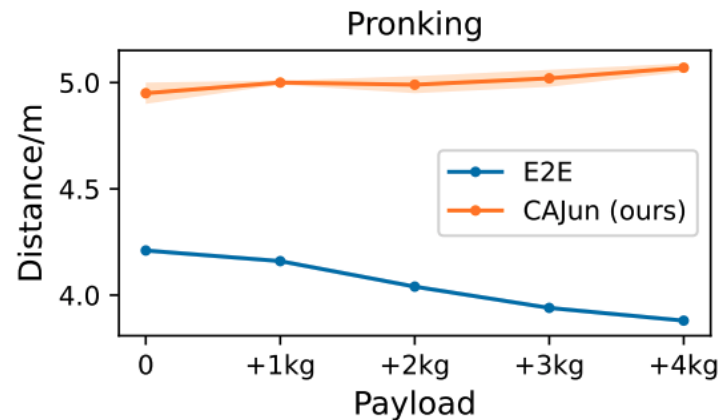
Learning Higher Level Policies

- ❑ Robust and adjustable continuous jumping



- ❑ Main advantages of CAJun (compared to end-to-end RL):

- Longer jumping distances
- More robust
- Data efficiency (20 min training)
- Little domain randomization



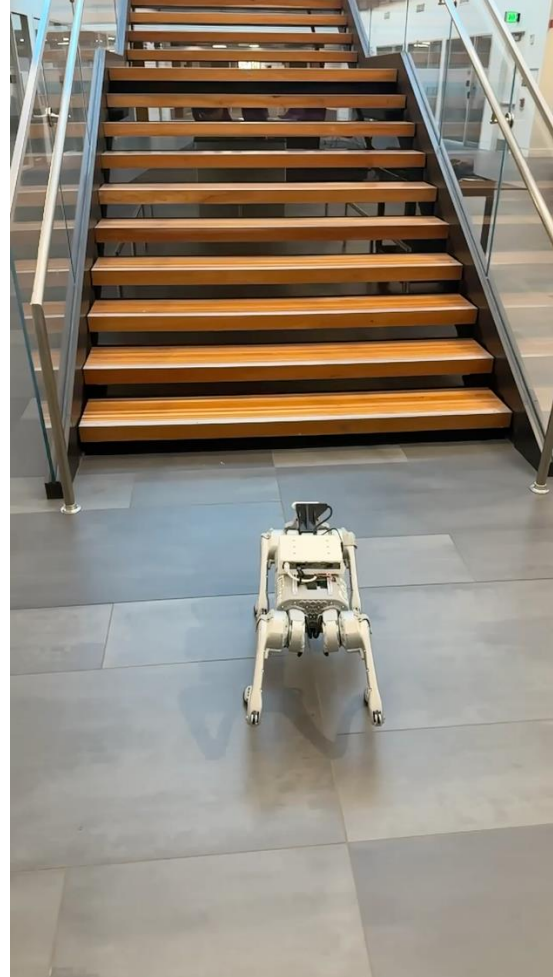
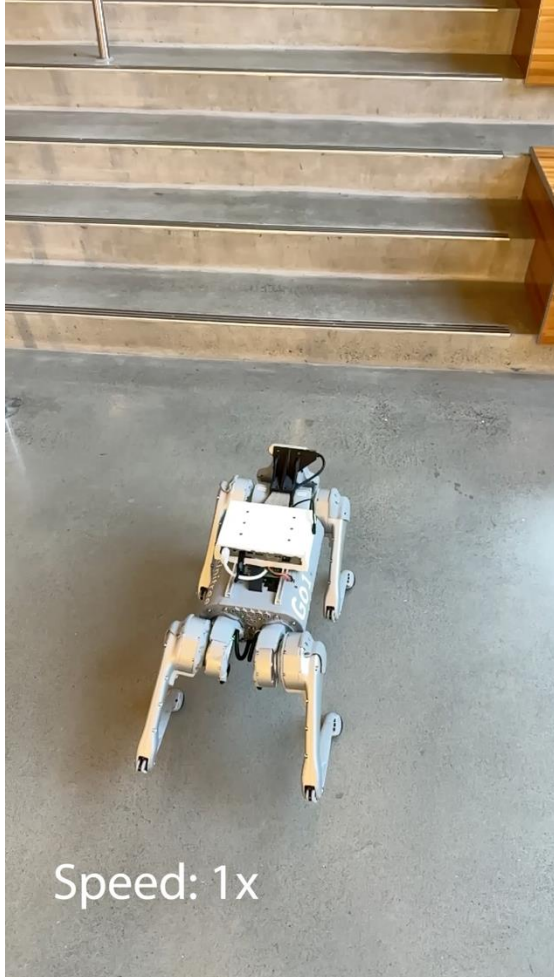
Learning Higher Level Policies



[Agile Continuous Jumping in Discontinuous Terrains, Yang et al., under review]

[CAJun: Continuous Adaptive Jumping using a Learned Centroidal Controller, Yang et al., CoRL'23]

Learning Higher Level Policies



	Robot	Style	Stairs (Max Speed)	Stepping Stones (Max Speed)	Step (Max Height)
Egocentric-Vision ^[15]	A1	Walking	0.46m/s	0.36m/s	0.26m (1×)
Robot Parkour ^[10]	A1	Jumping	-	-	0.4m (1.6×)
Extreme Parkour ^[9]	A1	Jumping	-	-	0.5m (2×)
Ours	Go1	Jumping	1m/s	1.4m/s	0.6m (2.4×)

[Agile Continuous Jumping in Discontinuous Terrains, Yang et al., under review]

[CAJun: Continuous Adaptive Jumping using a Learned Centroidal Controller, Yang et al., CoRL'23]

Learning Higher Level Policies

□ Benchmark for quadrotors

- LV: RL outputs linear velocity
- CTBR: total thrust and desired angular rate
- SRT: thrust for each rotor

A Benchmark Comparison of Learned Control Policies for Agile Quadrotor Flight

Elia Kaufmann, Leonard Bauersfeld, Davide Scaramuzza

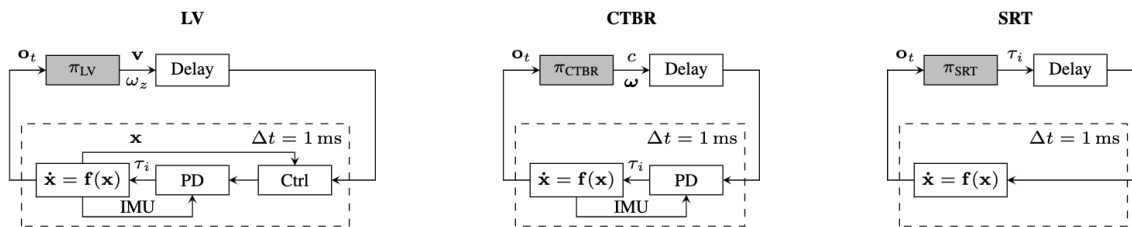


TABLE V: Average positional tracking error in centimeter on each test trajectory in case of no model mismatch. The table reports results for learned policies (SRT, CTBR, LV), and traditional approaches (MPC-SRT, MPC-CTBR). Results report mean and standard deviation for 10 trained policies.

	SRT	CTBR	LV	MPC-SRT	MPC-CTBR
Hover	1.0 ± 0.2	0.6 ± 0.2	7.0 ± 1.6	0.1	0.2
RandA	1.5 ± 0.2	0.9 ± 0.1	15.4 ± 3.0	0.2	0.3
RandB	2.4 ± 0.2	1.6 ± 0.1	61.5 ± 21.0	0.2	0.4
RandC	3.0 ± 0.3	2.0 ± 0.2	85.7 ± 11.5	0.2	0.4
RaceA	5.0 ± 1.2	5.0 ± 1.0	121.1 ± 25.8	0.3	1.3
RaceB	7.1 ± 1.8	6.9 ± 1.5	170.2 ± 16.3	0.7	3.0
Split-S	3.5 ± 0.4	6.6 ± 1.1	92.1 ± 20.8	1.0	2.1
RaceC	9.2 ± 3.2	12.3 ± 2.2	197.9 ± 38.1	1.2	3.9

TABLE VI: Average positional tracking error in centimeter on each test trajectory obtained in a quadrotor simulator based on blade-element momentum theory with a control delay of 20 ms. Results report mean and standard deviation for 10 trained policies.

	SRT	CTBR	LV	MPC-SRT	MPC-CTBR
Hover	11.3 ± 4.5	0.6 ± 0.5	6.7 ± 2.0	1.0	0.5
RandA	12.0 ± 4.0	1.2 ± 0.5	17.8 ± 1.4	3.3	1.1
RandB	14.4 ± 2.4	2.2 ± 0.8	57.0 ± 12.0	7.0	2.0
RandC	17.6 ± 5.9	2.6 ± 0.8	78.9 ± 13.4	8.5	2.6
RaceA	crash	5.6 ± 1.7	144.0 ± 20.1	12.6	4.8
RaceB	crash	10.0 ± 4.0	171.4 ± 17.0	crash	6.3
Split-S	crash	6.9 ± 2.6	83.8 ± 9.7	11.3	11.4
RaceC	crash	14.9 ± 5.5	176.7 ± 22.0	crash	7.5

Outline

- ❑ Introduce robot simulators
- ❑ Why is sim2real useful but challenging?

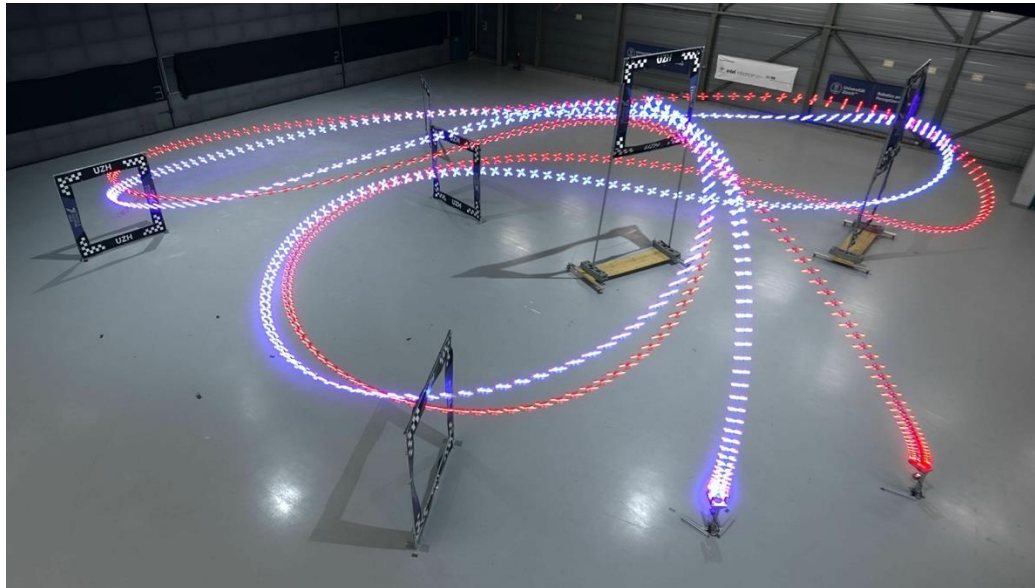
Methods to reduce the sim2real gap:

- ❑ Domain randomization
- ❑ Learning to adapt
- ❑ Learning higher-level policies
- ❑ Real2sim2real

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

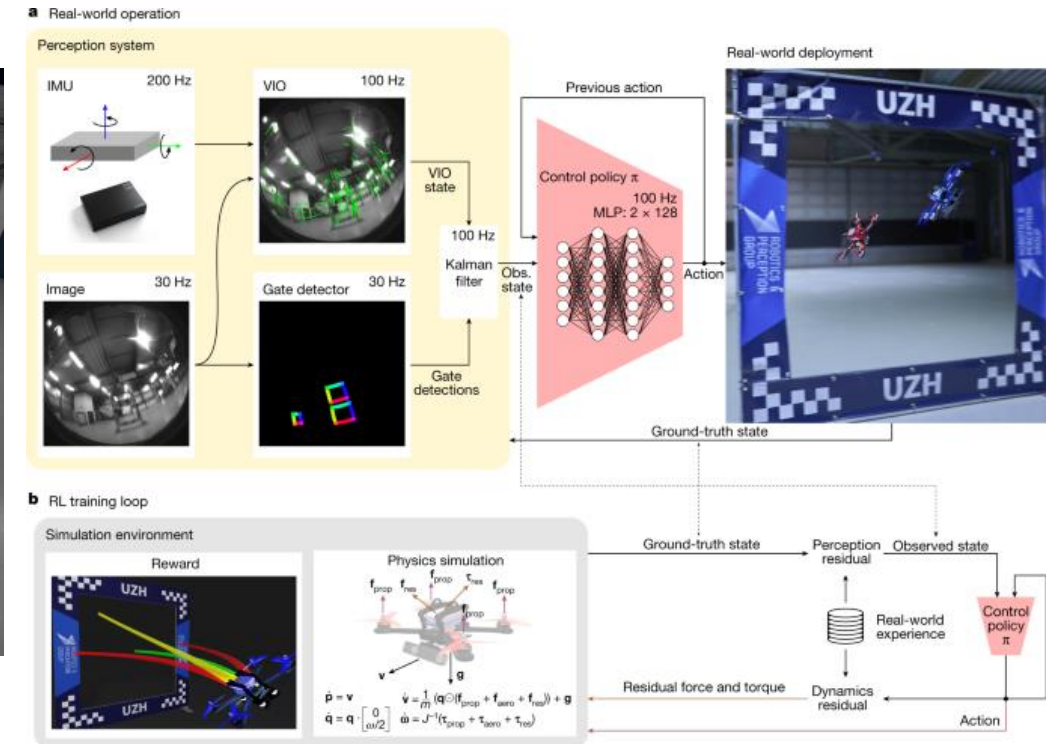
Real2Sim by Learning Perception/Dynamics Residuals

- ❑ Use real data to build/augment/improve a simulator, train DRL in simulation, deploy in the real world



Champion-level drone racing using deep reinforcement learning

[Elia Kaufmann](#) , [Leonard Bauersfeld](#), [Antonio Loquercio](#), [Matthias Müller](#), [Vladlen Koltun](#) & [Davide Scaramuzza](#)



Real2Sim by Learning Perception/Dynamics Residuals

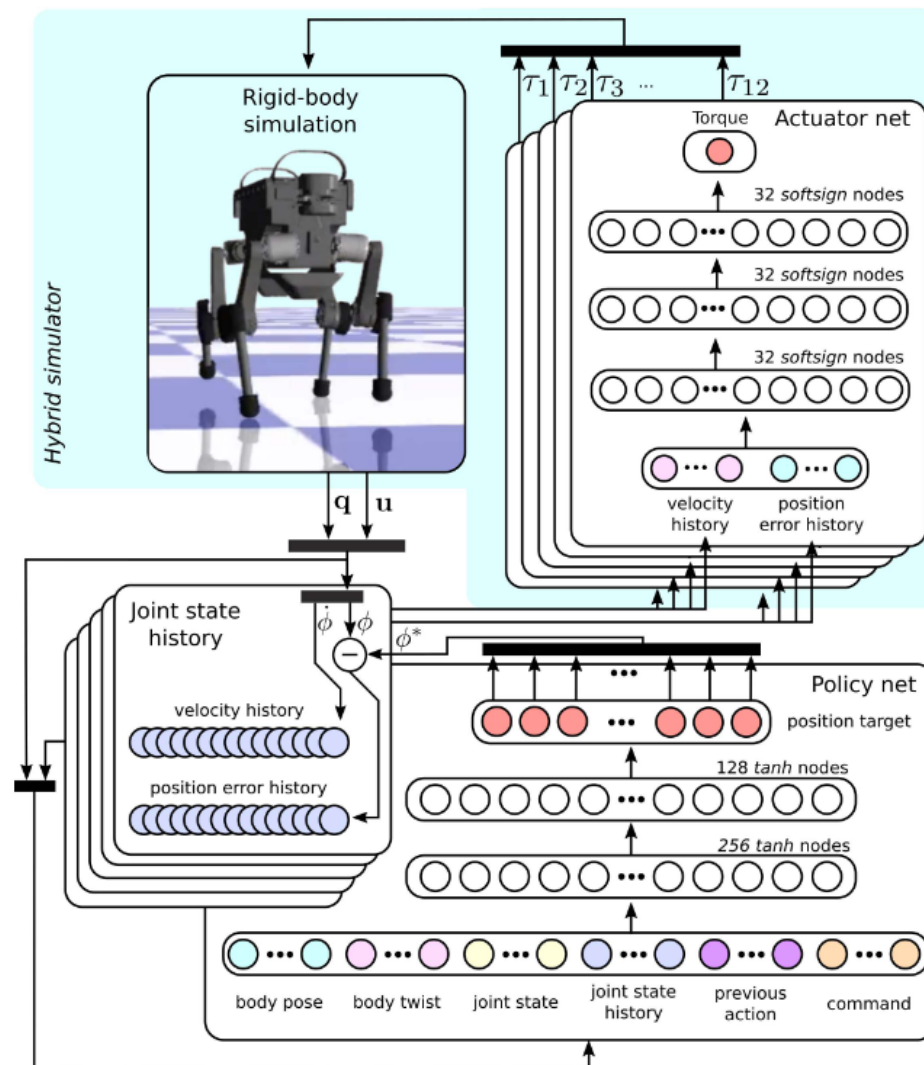
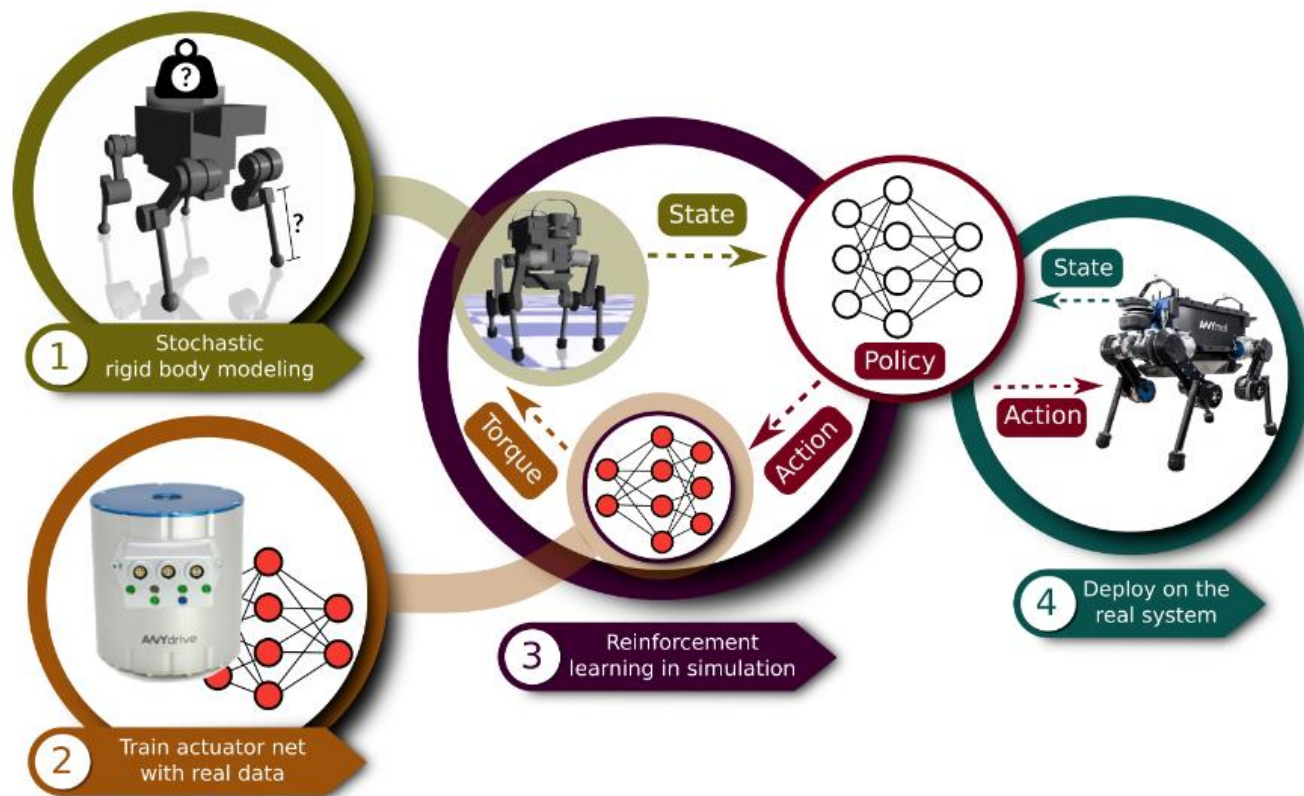
- ❑ Learn a simulator, train DRL in simulation, deploy in the real world



Actuator Net for Locomotion

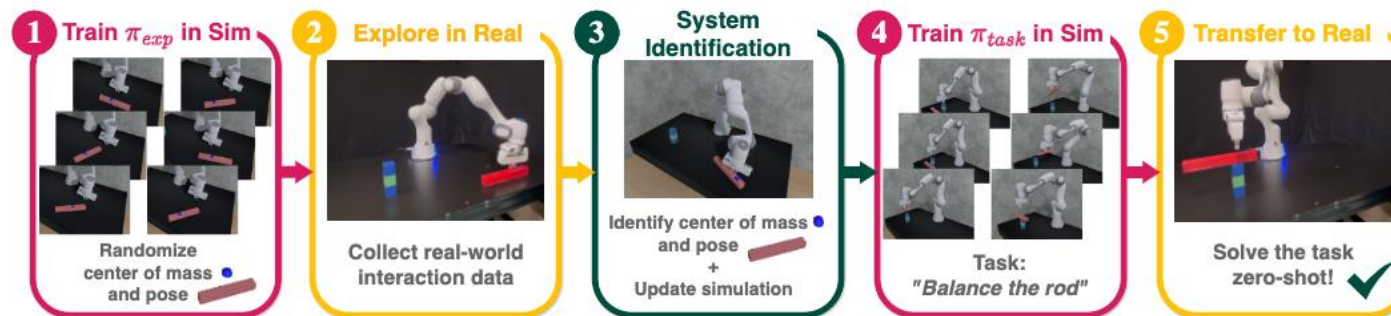
Learning agile and dynamic motor skills for
legged robots

Jemin Hwangbo^{1*}, Joonho Lee¹, Alexey Dosovitskiy², Dario Bellicoso¹, Vassilios Tsounis¹,
Vladlen Koltun³, Marco Hutter¹



Active Exploration for Real2Sim

- Actively collect data in the real world for Fisher information maximization



Implementing Fisher Information Maximization: In practice, several issues arise in solving (2), which we address here. First, the form of $\mathcal{I}(\theta, \pi)$ can be quite complicated, depending on the structure of $p_{\theta}(\cdot | \pi)$, and it may not be possible to efficiently obtain a solution to (2). To address this, we make a simplifying assumption on the dynamics, that our next state, s_{h+1} , evolves as:

$$s_{h+1} = f_{\theta}(s_h, a_h) + w_h, \quad (3)$$

where s_h and a_h are the current state and action, $w_h \sim \mathcal{N}(0, \sigma_w^2 \cdot I)$ is Gaussian process noise, and f_{θ} are the nominal dynamics. Under these dynamics, the Fisher information matrix reduces to

$$\mathcal{I}(\theta, \pi) = \sigma_w^{-2} \cdot \mathbb{E}_{p_{\theta}(\cdot | \pi)} \left[\sum_{h=1}^H \nabla_{\theta} f_{\theta}(s_h, a_h) \cdot \nabla_{\theta} f_{\theta}(s_h, a_h)^{\top} \right].$$

Thank You!